Assignment 2

Bayesian Networks and Neural Networks

Deadline: October 31st, 10:00 PM.
Total: 100 points.

Assignment Instructions:

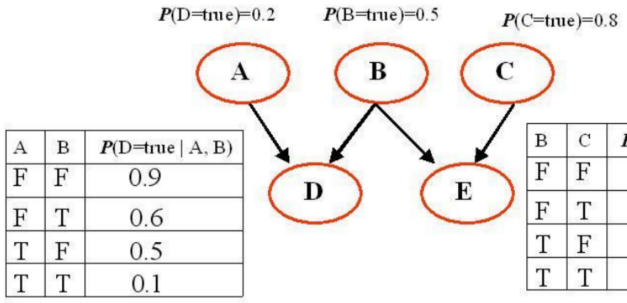
Teams: Work in teams of up to three students, or individually if you prefer. Working individually does not earn extra credit. Add every member of your team to your submission.

Submission Rules: This assignment comes with a Jupyter notebook, `hw2.ipynb`. Write your code and your answers in the notebook cells marked `# GRADED`, and submit the notebook through Gradescope. Each team should submit one notebook. Each question gives the name of its answer variable or function in `typewriter font`. Problem 3 needs PyTorch, NumPy and matplotlib (`pip install torch numpy matplotlib`); you can also use Google Colab, where all three are installed.

Late Submissions: Late assignments receive 0 points.

Collusion, Plagiarism, etc.: Each team must prepare its solutions independently from other teams, i.e., without using common notes, code or worksheets with other students or trying to solve problems in collaboration with other teams. You must indicate any external sources you have used in the preparation of your solution. Do not plagiarize online sources and in general make sure you do not violate any of the academic standards of the department or the university. Failure to follow these rules may result in failure in the course.

Problem 1 (*15 points):* Consider the following Bayesian network, where variables A through E are Boolean. The image has a typo: $P(D = true) = 0.2$ should read $P(A = true) = 0.2$.



A	B	P(D=true | A, B)
F	F	0.9
F	T	0.6
T	F	0.5
T	T	0.1

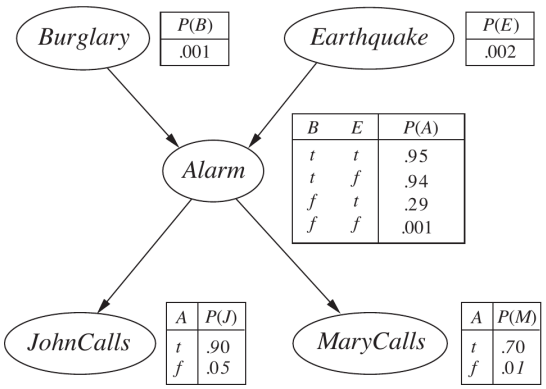
B	C	P(E=true | B, C)
F	F	0.2
F	T	0.4
T	F	0.8
T	T	0.3

(a) What is the probability that all five variables are true? (`P1A`, 5 points) [Hint: Use the network structure to write the joint probability as a product of the given probabilities.]

(b) What is the probability that all five variables are false? (`P1B`, 5 points) [Hint: Use the same approach as in (a).]

(c) What is the probability that A is false given that the four other variables are all known to be true? (`P1C`, 5 points)

Problem 2 (*20 points):* Consider the "burglary" Bayesian network below.



(a) Calculate $P(Burglary \mid JohnCalls = true, MaryCalls = true)$ (`P2A`, 5 points). To do the calculation, write the function `enumeration_ask(query, evidence, network)`, which implements the enumeration algorithm from the book (10 points).

(b) Suppose a Bayesian network has the form of a *chain*: a sequence of Boolean variables $X_1, \ldots, X_n$ where $Parents(X_i) = \{X_{i-1}\}$ for $i = 2, \ldots, n$. What is the complexity of computing $P(X_1 \mid X_n = true)$ using enumeration? (`P2B`, 5 points)

Choose the answer to (b) from:

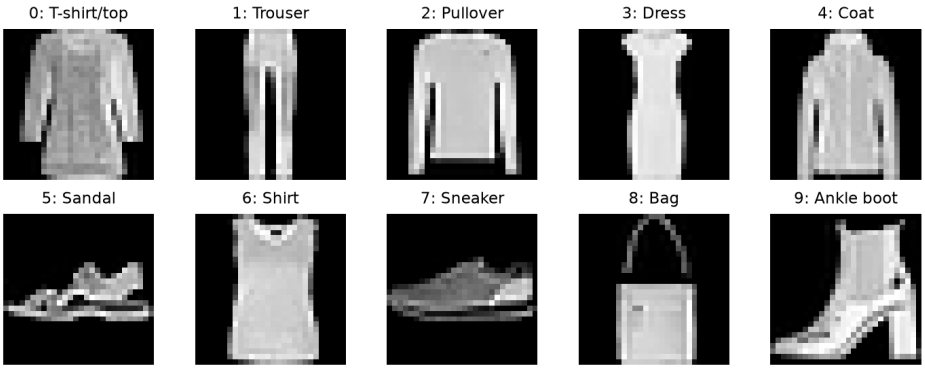
(A) $O(n)$

(B) $O(n^2)$

(C) $O(2^n)$

(D) $O(n!)$

Problem 3 (*65 points):* Build a multilayer perceptron (MLP) with PyTorch to recognize clothing in Fashion-MNIST, a dataset from Zalando Research. Each example is a 28 by 28 grayscale image with one of 10 labels, shown below. The input is the image flattened into 784 pixel values; the model outputs 10 class scores. The starter includes 54,000 training images and 6,000 validation images, along with functions to load the data and display images, predictions and training loss.
For help with PyTorch, read Build the Neural Network and Optimizing Model Parameters. These tutorials also use Fashion-MNIST. Additional reference: Fashion-MNIST video tutorials. The notebook includes a small example to get you started.



(a) Write a class `MLP(in_features, hidden_sizes, out_features, activation)` that subclasses `torch.nn.Module`. Build it from `nn.Linear` layers and activation modules: one `nn.Linear` layer for each hidden layer, each followed by an activation (`nn.ReLU`, `nn.Tanh`, `nn.Sigmoid`, `nn.LeakyReLU` or `nn.Identity`, chosen by `activation` = "relu", "tanh", "sigmoid", "leaky_relu" or "identity"), and a final `nn.Linear` output layer with no activation. The `forward` method returns the class scores. Support any input and output sizes, and an empty list of hidden layers. (15 points)

(b) Write `train_model(model, X, y, epochs, lr, batch_size, seed)`, which trains the model with mini-batch Adam and cross-entropy loss and returns the average training loss of each epoch (10 points). Also write `predict(model, X)`, which returns the predicted class of each example (5 points). The docstrings in the notebook give the details.

(c) Compare different hidden layers, layer sizes, activation functions and training settings using validation accuracy. Inspect the mistakes your models make. Then write `build_and_train(X_train, y_train, X_val, y_val)`, which builds your best model, trains it and returns it. Train on the training split only; use validation data to choose settings. Use only the supplied data and train from scratch. Convolutional layers and pretrained models are not allowed. Your model is evaluated on the 10,000 official test images. A test accuracy of 0.87 or more earns the full 20 points, 0.80 or less earns 0 points, and scores in between earn proportional credit. The model may have at most 2 million parameters, and `build_and_train` must finish within 5 minutes on a CPU using one thread. (20 points)

(d) Answer the following questions (3 points each).

(1) With no hidden layers (`hidden_sizes = []`), each class score is (`P3D1`): (A) always between 0 and 1; (B) a weighted sum of the 784 pixel values plus a bias; (C) the index of the brightest pixel; (D) a product of the pixel values.

(2) A network with three hidden layers and only "identity" activations is equivalent to (`P3D2`): (A) a nearest-neighbor classifier; (B) a single linear layer with a bias; (C) a ReLU network; (D) a different kind of function for each choice of layer sizes.

(3) As a function of the pixel values, the class scores of a ReLU MLP are (`P3D3`): (A) always probabilities; (B) always constant; (C) smooth everywhere; (D) piecewise linear.

(4) A model has a training accuracy of 0.99 and a validation accuracy of 0.85. This is a sign of (`P3D4`): (A) overfitting; (B) underfitting; (C) a learning rate that is too small; (D) an error in the validation data.

(5) Deep networks with sigmoid activations often train more slowly than with ReLU activations. Why? (`P3D5`) (A) The sigmoid takes longer to compute. (B) The sigmoid can be negative. (C) The derivative of the sigmoid is at most 1/4, so the gradients can shrink from layer to layer. (D) The sigmoid makes the network linear.